

第13章 模块系统与构建工具

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解模块系统(JPMS)的动机:封装、显式依赖、可伸缩
- 掌握 module-info.java:requires / exports / opens 的语义
- 掌握 jar 的打包、运行与 classpath / module-path 的区别
- 掌握 Maven 的项目结构与核心命令
- 掌握 pom.xml 的依赖声明与依赖传递机制
- 理解依赖冲突的产生与解决策略
- 能完成"编写 → 构建 → 打包 → 运行"的完整流程

💡 从 C 到 Java:把"工程"组织起来

前 12 章都在写"单文件程序",生产代码不是这样:成百上千个类、几十个第三方库、明确的版本要求。C 有 makefile/CMake + 静态库 + 头文件;Java 有 Maven/Gradle + jar + 模块系统。对照关系:jar ≈ .a/.so 库;Maven 仓库 ≈ 系统包管理器(apt);pom.xml ≈ CMakeLists + 依赖清单;module-info.java ≈ 头文件里的"导出了什么"。

模块系统(JPMS,JDK9)回答三个问题。① 我的包向谁公开?(exports,像头文件的 #include 白名单);② 我需要谁的包?(requires,像链接时的 -lxxx);③ 谁可以反射我的包?(opens,第11章 setAccessible 的开关)。在模块化世界里,封装从"类级别"提升到"模块级别"——不想让外部碰的包,根本不导出,编译期就挡在门外。

Maven 是现代 Java 的门槛。不学 Maven,连 Spring Boot 项目都跑不起来。核心心智:"坐标"定位依赖(groupId:artifactId:version),中央仓库下载,约定优于配置的目录结构,pom.xml 声明一切。C 程序员刚接触时最不适应的:不是"怎么编译"(IDE 代劳),而是"依赖从哪来、版本怎么定、冲突怎么办"。

构建生命周期是流水线。compile → test → package → install,一步一个产物,像 makefile 的 target 依赖,但约定固定、顺序固定。CI/CD(第14章提)就是在这条流水线上加步骤。

学习顺序:模块系统(§13.1-13.3)建立"模块化思维",jar 打包(§13.3),Maven(§13.4-13.6),依赖管理(§13.7)。

📦 前置知识自查

- 第11-12章:反射与注解(opens 指令直接管反射权限)
- C 的编译组织:makefile、静态/动态库、头文件、链接过程
- 命令行基本操作:javac/java 已熟练(第1章)

🚀 本章学习路线

1. **第一阶段(约 1 小时):**§13.1-13.2 模块化的动机与 module-info 三指令
2. **第二阶段(约 1.5 小时):**§13.3 手工 jar 打包与 module-path 运行,体会"模块边界"
3. **第三阶段(约 2 小时):**§13.4-13.7 Maven 结构与依赖管理,完成章末实验
4. **验收标准:**能写出 module-info 三指令;能解释 classpath 与 module-path;能读懂 pom.xml 并说出依赖冲突解法

本章知识导图

- §13.1 从 classpath 到模块系统
- §13.2 module-info.java:requires / exports / opens
- §13.3 jar 打包与运行:classpath vs module-path
- §13.4 Maven 项目结构与约定
- §13.5 pom.xml:坐标、依赖、插件
- §13.6 构建生命周期与命令
- §13.7 依赖管理:传递依赖与冲突

§13.1 从 classpath 到模块系统

13.1.1 传统 classpath 的三个问题

JDK9 之前,Java 把 jar 都塞进 classpath: ① 没有封装边界 (public 类谁都能用); ② 依赖关系隐式 (谁 require 谁只有运行期 ClassNotFoundException 才知道); ③ 无法裁剪运行时 (JDK 全量加载)。模块系统(Project Jigsaw)从 JDK9 开始根治:每个模块显式声明 requires/exports,JDK 自身也模块化。

概念	C 的世界	Java 的世界
代码单元	.c/.h 文件	.java 文件(包组织)
编译单元	目标文件 .o	.class 字节码
库	.a / .so / .dll	jar(zip 打包的 class)
导出接口	头文件 #include 白名单	module-info 的 exports(模块化时)
链接依赖	-lxxx / ld 脚本	requires(模块化)/ Maven 依赖
包管理器	apt / brew / vcpkg	Maven Central(仓库)
构建脚本	Makefile / CMakeLists	pom.xml(Maven)/ build.gradle

★ 重点:模块化是"可选"的——但理解它必须

绝大多数业务项目至今跑在 classpath 上(模块化要求严格、第三方库参差),但模块系统是 JDK 的地基、是 Spring/框架"开箱即用"的保证,也是面试与排障的常识。学习策略:会写 module-info(小模块项目)、会解释三指令、知道反射需要 opens——生产大项目里模块化交给架构师决策。

§13.2 module-info.java:三个指令

13.2.1 最小模块声明

示例 13-1 module-info 三指令

```
// 文件位置:模块根目录下的 module-info.java(与包目录平级)
module com.example.students {
    // requires:本模块需要谁(相当于 C 的 #include + 链接)
    requires java.logging;

    // exports:把哪个包公开给外部(相当于头文件公开 API)
    exports com.example.students.api;

    // opens:允许哪个包被反射访问(第11章 setAccessible 的前提)
    // 模块化世界里,反射别人需要显式授权
    opens com.example.students.internal;

    // 需要反射 JDK 内部包时(极少见,通常意味着设计问题):
    // requires jdk.unsupported;
}
```

示例 13-2 模块化的小项目

```
// 目录结构:
// mods/
//   com.example.calc/
//     module-info.java
//     com/example/calc/Calculator.java      (public,导出)
//     com/example/calc/InternalCache.java   (public,但不导出:外部不可见)

// ---- module-info.java ----
module com.example.calc {
    exports com.example.calc;      // 只导出这个包
}

// ---- Calculator.java ----
package com.example.calc;

public class Calculator {
    // 公开 API:外部可调用
    public static int add(int a, int b) { return a + b; }

    public static void clearCache() {
        InternalCache.reset();      // 模块内部可自由访问
    }
}

// ---- InternalCache.java(不导出:外部 import 直接编译错误)----
package com.example.calc;

public class InternalCache {
    static void reset() { System.out.println("缓存已清空"); }
}
```

⚠ 易错点 1:模块化后"外部用不了"是编译错误,不是运行错误

模块化世界里,不导出的包被外部 import 时 编译直接失败 (package is not visible)——比 classpath 时代"运行期 ClassNotFoundException"早得多,这正是模块化的价值:封装在编译期兑现。同理,反射未 opens 的包,setAccessible 抛 InaccessibleObjectException(JDK9+),这也是第11章"安全裂缝"的官方补丁。

§13.3 jar 打包与运行

13.3.1 手工流程:编译 → 打包 → 运行

示例 13-3 手工 jar 全流程(命令行)

```
# 1. 普通编译(未模块化):classpath 方式
javac -d out src/com/example/app/*.java
# 2. 打 jar(相当于把 .o 归档成静态库)
jar --create --file app.jar -C out .
# 3. 运行(classpath 方式)
java -cp app.jar com.example.app.Main

# ---- 模块化流程 ----
# 1. 编译模块(multi-release 语法按需;这里普通编译)
javac --module-path mods -d out \
    src/com.example.calc/module-info.java \
    src/com.example.calc/com/example/calc/*.java
# 2. 打模块 jar(带 module-info 的 jar 就是模块)
jar --create --file mods/com.example.calc.jar -C out .
# 3. 用 module-path 运行(--module-path 替代 -cp,module 名.主类)
java --module-path mods --module com.example.calc/com.example.calc.Calculator

# 查看模块描述:
jar --describe-module --file mods/com.example.calc.jar
```

★ 重点:classpath 与 module-path 的区别

-cp(classpath):一堆 jar 平铺,无边界、无依赖声明,类找到了算数;--module-path:模块化 jar 的集合,依赖由 module-info 声明,JVM 启动时验证。判断一个 jar 是"模块"还是"普通 jar":jar 根目录有没有 module-info.class。绝大多第三方库是普通 jar(自动模块名 = 文件名去版本),这就是"模块化推进慢"的根源——但语法与原理不变。

§13.4 Maven 项目结构

13.4.1 约定优于配置

示例 13-4 Maven 标准目录

```
myapp/
├── pom.xml                # 项目描述:坐标/依赖/插件(唯一需要手写的)
├── src/
│   ├── main/
│   │   ├── java/         # 生产代码
│   │   │   └── com/example/app/Main.java
│   │   └── resources/    # 配置文件(properties 等,自动进 classpath)
│   └── test/
│       └── java/         # 测试代码(第14章)
├── target/               # 构建产物(自动生成,不提交 git)
│   ├── classes/         # 编译后的 class
│   └── myapp-1.0.jar     # 打包产物
```

问题	C 的回答	Maven 的回答
目录怎么排	自己定,CMake 里配	约定:src/main/java 等,零配置
依赖从哪来	apt 装系统库 / 手动放	pom.xml 声明,中央仓库自动下载
怎么编译	make / cmake --build	mvn compile
怎么打包	make install(拷 .so)	mvn package(产出 jar)
测试怎么跑	自己写脚本	mvn test(JUnit 集成)

§13.5 pom.xml:坐标、依赖、插件

示例 13-5 最小可用 pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <!-- 坐标:仓库里唯一标识(类比 apt 包名 + 版本) -->
  <groupId>com.example</groupId>           <!-- 组织(反向域名) -->
  <artifactId>myapp</artifactId>           <!-- 项目名 -->
  <version>1.0.0</version>                 <!-- 版本号 -->
  <packaging>jar</packaging>

  <!-- 属性:统一版本,避免散落 -->
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <!-- 依赖:坐标三段 + 可选 scope -->
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.17.1</version>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>                 <!-- 只进测试 -->
    </dependency>
  </dependencies>
</project>

```

⚠ 易错点 2:scope 的作用域

scope 决定依赖"活到哪一步":compile(默认,打包进 jar)、test(仅测试)、provided(编译需要但运行时由容器提供,如 servlet-api)、runtime(运行需要编译不需要)。经典错误:把测试库(如 JUnit)声明成 compile,产出的 jar 膨胀几 MB,还可能在生产触发类冲突。记忆:依赖属于谁的使用期,就标准的 scope。

§13.6 构建生命周期与命令

13.6.1 一条命令链,逐级递进

示例 13-6 核心命令速查

```

mvn clean           # 删除 target/ 目录(干净的起点)
mvn compile         # 只编译 src/main/java → target/classes
mvn test            # 编译 + 运行测试(JUnit)
mvn package         # 编译 + 测试 + 打包 jar(到 target/)
mvn install         # package + 安装进本地仓库(~/.m2),供其他项目依赖
mvn dependency:tree # 查看依赖树(排查冲突的第一工具)
mvn help:effective-pom # 看"实际生效"的 pom(继承父 pom 之后)

# 跳过测试(不推荐,紧急情况下才用):
mvn package -DskipTests

# 指定 Java 版本:
mvn package -Dmaven.compiler.release=17

```

🚀 工程建议:Maven 的三个日常习惯

1. 每次提交前 `mvn test`:测试不过的代码不进仓库(CI 也会拦,第14章)
2. `target/` 不提交 `git .:gitignore` 写死,产物人人可重建
3. 依赖版本统一管理:多模块项目用 `dependencyManagement` 或 BOM(bill of materials)集中管版本,别让各模块各写各的

§13.7 依赖管理:传递依赖与冲突

13.7.1 传递依赖与"最近优先"

你的项目依赖 A,A 依赖 B——B 自动进入你的 classpath,这叫**传递依赖**(像 apt 自动装依赖包)。冲突:A 要 B 1.0,C 要 B 2.0,谁赢? **Maven 默认"最近优先 + 先声明优先"**:依赖树中离根最近的版本胜出;距离相同先声明者胜出。这个"自动决策"经常选错,所以 `mvn dependency:tree` 是每个 Java 工程师的日常工具。

示例 13-7 依赖冲突排查实录

```
# 症状:NoSuchMethodError / NoClassDefFoundError(编译正常、运行爆炸)
# 典型原因:classpath 上同时存在库的两个版本,方法签名对不上

# 1. 看依赖树,找出同一个库的不同版本:
mvn dependency:tree
# [INFO] com.example:myapp:jar:1.0.0
# +- com.example:lib-a:jar:1.0
# |   \- com.fasterxml.jackson.core:jackson-databind:jar:2.17.1
# \- com.example:lib-c:jar:1.0
#     \- com.fasterxml.jackson.core:jackson-databind:jar:2.16.0 ← 冲突!

# 2. 解法一:显式声明想要的版本(最近优先,你的声明必赢)
# 在 pom.xml 的 dependencies 里直接加:
# <dependency>
#   <groupId>com.fasterxml.jackson.core</groupId>
#   <artifactId>jackson-databind</artifactId>
#   <version>2.17.1</version>
# </dependency>

# 3. 解法二:排除某个传递依赖(明确不要它)
# <dependency>
#   <groupId>com.example</groupId>
#   <artifactId>lib-c</artifactId>
#   <version>1.0</version>
#   <exclusions>
#     <exclusion>
#       <groupId>com.fasterxml.jackson.core</groupId>
#       <artifactId>jackson-databind</artifactId>
#     </exclusion>
#   </exclusions>
# </dependency>
```

⚠ 易错点 3:NoSuchMethodError 是"版本冲突"的代名词

编译期一切正常,运行期抛 `NoSuchMethodError` / `NoClassDefFoundError` / `AbstractMethodError`:先别怀疑代码,十有八九是 classpath 上同一库的两个版本并存。排查顺序:① `mvn dependency:tree` 找重复;② 显式声明正确版本;③ 排除老版本来源;④ 再不行查 jar 内容 `jar tf`。C 的"链接错版本 .so"是同一个坑,只是 Java 的报错更隐晦(运行时才爆)。

本章速查表

主题	要点
模块化动机	封装边界 / 显式依赖 / 运行时可裁剪;JDK9+ 的 Jigsaw
requires	声明"我需要谁的模块";编译期检查,缺失立即报错
exports	公开哪些包给外部;没导出的包外部 import 编译失败
opens	允许哪些包被反射;反射未 opens 抛 InaccessibleObjectException
jar	zip 打包 class:jar --create --file x.jar -C out .;module-info 在根目录
classpath vs module-path	-cp 平铺无边界;--module-path 模块化,启动时验证依赖
Maven 目录	pom.xml + src/main/java + src/test/java + target/(产物)
坐标	groupId:artifactId:version 三段定位;仓库: Maven Central
scope	compile(默认)/ test / provided / runtime;选错 jar 膨胀或运行缺类
生命周期	clean → compile → test → package → install 逐级递进
传递依赖	A 依赖 B 自动带入;冲突"最近优先+先声明优先"
冲突排查	mvn dependency:tree → 显式版本 → 排除;NoSuchMethodError = 版本冲突信号

最小必会清单

- 能说出模块化解的三个问题,并解释 exports/requires/opens
- 能写出 module-info.java 的最小声明并解释"编译期封装"
- 能用命令行完成 javac → jar → java 的完整流程
- 能说出 classpath 与 module-path 的区别
- 能画出 Maven 标准目录并解释每个目录的用途
- 能读懂 pom.xml:坐标、依赖、scope、properties
- 能说出 mvn 五个核心命令与 dependency:tree 的用途
- 能解释传递依赖与"最近优先"规则,会排查 NoSuchMethodError

自测题

Q 1. 模块 A 的 module-info 只写了 exports com.a.api,但代码里引用了 com.a.internal 的类,外部模块能编译通过吗?

✓ 参考答案

不能。外部模块 import com.a.internal 时编译报 "package is not visible"。即使两个类都是 public——模块边界比访问修饰符更严:public 决定"类内可见性上限",exports 决定"模块外可见性上限",取交集。这就是模块化时代的封装模型。

Q 2. 模块化项目里,反射代码访问另一个模块的类抛 InaccessibleObjectException,怎么解决?顺序是什么?

✓ 参考答案

① 最佳:找提供方,让它在自己模块的 module-info 加 opens com.example.internal; (或 exports,如果本来就该公开);② 次优:自己写"传递 opens"——被反射的包在提供方必须 open;③ 绕行:反射方的模块加 --add-opens JVM 参数 (临时方案,生产慎用)。永远先问"这个反射是否必要",再谈 opens——反射他人内部是最后手段。

Q 3. -cp 与 --module-path 的区别?普通 jar 能在 --module-path 上运行吗?

✓ 参考答案

-cp:平铺的 jar 集合,运行时按需查找类,无依赖声明。--module-path:模块 jar 集合,JVM 启动时解析 module-info 验证依赖关系。普通 jar(无 module-info)可以放 module-path——它被当成"自动模块",模块名取 jar 文件名(去版本号)。所以"模块化迁移"可以渐进:先把普通 jar 放 module-path 观察,再逐个补 module-info。

Q 4. 你的项目同时被 lib-a(依赖 Jackson 2.17)与 lib-c(依赖 Jackson 2.16)依赖,Maven 会选哪个版本?为什么这可能是错的?

✓ 参考答案

看依赖树深度:若两者都是直接传递一层,按声明顺序先声明者胜;若 lib-a 在更浅的层级,2.17 胜。自动决策可能错:比如 lib-c 只与 2.16 兼容,或更浅层级反而不是"最想要"的版本。所以冲突必须人工复核:显式声明目标版本,必要时排除,并跑完整测试验证兼容性。

Q 5. 运行期抛 NoSuchMethodError,代码却编译通过。为什么?第一步做什么?

✓ 参考答案

编译用了一版 API,运行 classpath 上却是另一版(旧版没有该方法,或签名不同)。这是依赖版本冲突的典型信号,不是你的代码 bug。第一步:mvn dependency:tree 找同 artifactId 的多版本;然后显式锁定正确版本,排除错误来源,重跑测试验证。

Q 6. 写一段文字解释 Maven 生命周期:为什么 mvn package 会自动编译、会自动跑测试?

✓ 参考答案

Maven 生命周期是固定流水线:validate → compile → test → package → verify → install,后一阶段自动包含前面所有阶段。mvn package = 执行到 package 为止,所以编译与测试是必经环节。这也解释了为什么"只想打包不想测试"要显式 -DskipTests——默认行为就是全流程。这与 makefile 的"目标依赖"等价,只是顺序被约定死、无法打乱。

章末实验题:模块化计算器 + Maven 构建

实验 13:模块化计算器与 Maven 构建

实验目标:完成两个递进任务:① 手工构建一个模块化计算器(module-info + jar + module-path 运行);② 用 Maven 构建一个带第三方依赖(Jackson 解析 JSON 配置)的项目。综合运用本章全部知识点。

实验要求:

- 任务 1:创建模块 com.example.calc,导出 api 包,内部实现包不导出(覆盖 §13.1-13.2 exports/requires)
- 任务 2:模块内用 java.util.logging 记录计算日志,module-info 加 requires java.logging(覆盖 §13.2 requires)
- 任务 3:手动执行 javac --module-path → jar --create → java --module-path 三条命令,输出计算器结果(覆盖 §13.3 打包运行)
- 任务 4:写一个"非法访问验证":在模块外 import 未导出包,观察编译错误(覆盖 §13.2 封装)
- 任务 5:创建 Maven 项目结构(手写 pom.xml 与目录),声明 jackson-databind 依赖,代码从 JSON 文件读配置(覆盖 §13.4-13.5)
- 任务 6:执行 mvn package 产出 jar,用 java -cp target/classes:依赖路径 或 mvn exec 运行,输出计算结果(覆盖 §13.6 构建)
- 任务 7:运行 mvn dependency:tree,解释传递依赖(覆盖 §13.7 依赖)

实验环境:JDK 17+(自带 jar 工具);Maven 3.8+(若未安装:任务 5-7 可用"手工下载 jar + javac -cp"替代,命令差异在参考答案注释中说明)。

第一步 做任务 1-4(纯命令行,无外部依赖),成功后再进 Maven 部分

第二步 务 3 的目录结构:mods/com.example.calc/{module-info.java, com/example/calc/...}

第三步 务 5 的 JSON 文件:config.json 放 src/main/resources,读类用 getResourceAsStream

参考答案要点

```
// ===== 任务 1-4:模块化计算器(手工命令行) =====
// 目录:mods/com.example.calc/
// module-info.java
// com/example/calc/Calculator.java
// com/example/calc/LogUtil.java      (不导出)

// ---- module-info.java ----
module com.example.calc {
    requires java.logging;           // 用 JDK 的日志模块
    exports com.example.calc;       // 只导出公共包
}

// ---- Calculator.java ----
package com.example.calc;

import java.util.logging.Logger;

public class Calculator {
    private static final Logger LOG = Logger.getLogger("calc");

    public static int add(int a, int b) {
        LOG.info("计算: " + a + " + " + b);
        return a + b;
    }

    public static int sub(int a, int b) {
        LOG.info("计算: " + a + " - " + b);
        return a - b;
    }
}

// ---- LogUtil.java(不导出:包 com.example.calc 整体导出了?)
// 注意:exports com.example.calc 会导出该包全部 public 类,
// 想"部分隐藏"应拆包—此处 LogUtil 用包私有访问演示内部依赖:
package com.example.calc;

class LogUtil {                      // 包私有:模块内外都不可见
    static String tag(String s) { return "[calc] " + s; }
}

// ---- 主程序(模块外,只依赖导出 API)----
// 目录:mods/app/com/example/app/Main.java
package com.example.app;

import com.example.calc.Calculator;    // 合法:导出包
// import com.example.calc.LogUtil;    // 编译错误:不可见(任务 4 验证)

public class Main {
    public static void main(String[] args) {
        System.out.println("3 + 4 = " + Calculator.add(3, 4));
        System.out.println("9 - 2 = " + Calculator.sub(9, 2));
    }
}

// ---- 任务 3:命令行三条命令 ----
// javac --module-path mods -d out/calc mods/com.example.calc/module-info.java \
//     mods/com.example.calc/com/example/calc/*.java
// jar --create --file mods/com.example.calc.jar -C out/calc .
// javac --module-path mods -d out/app mods/app/com/example/app/Main.java
// java --module-path mods --module app/com.example.app.Main

// ===== 任务 5-7:Maven + Jackson(任务 5-6) =====
// pom.xml(见示例 13-5,加 jackson-databind 2.17.1)
// src/main/resources/config.json:
// { "a": 10, "b": 5 }

// ---- JsonConfigApp.java ----
package com.example.app;
```

```

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

import java.io.InputStream;

public class JsonConfigApp {
    public static void main(String[] args) throws Exception {
        // 从 classpath 读 resources 下的配置文件
        ObjectMapper mapper = new ObjectMapper();
        try (InputStream in = JsonConfigApp.class
            .getResourceAsStream("/config.json")) {
            JsonNode cfg = mapper.readTree(in);
            int a = cfg.get("a").asInt();
            int b = cfg.get("b").asInt();
            System.out.println(a + " + " + b + " = " + (a + b));
            System.out.println(a + " * " + b + " = " + (a * b));
        }
    }
}

// ---- 任务 6:构建与运行 ----
// mvn package
// java -cp target/classes:$HOME/.m2/repository/com/fasterxml/jackson/... \
//     com.example.app.JsonConfigApp
// (Windows 用分号分隔 classpath;Maven 的 exec 插件可简化:
// mvn exec:java -Dexec.mainClass=com.example.app.JsonConfigApp)

// ---- 任务 7:依赖树(节选) ----
// mvn dependency:tree
// jackson-databind → jackson-core + jackson-annotations(传递依赖)
// 若多个库各带不同版本 jackson,按"最近优先"规则解析,见 §13.7

```

设计说明:任务 2 的 `java.logging` 是 JDK 自带模块——requires 它演示了"依赖 JDK 模块"的标准写法;任务 4 用"取消注释即编译失败"让学生亲手验证模块封装;任务 5-6 的 Jackson 是生产最常用的 JSON 库,`getResourceAsStream` 读 `resources` 是标准姿势(注意 Windows 的 `classpath` 分隔符是分号)。没有 Maven 的环境,任务 5-7 可改为:从 Maven 中央仓库手动下载 `jackson-databind/core/annotations` 三个 jar,用 `javac -cp` 与 `java -cp` 完成等价流程。扩展方向:给模块加 `opens` 演示反射授权(与第11章衔接)、`pom` 加 `surefire` 插件跑 JUnit 测试(与第14章衔接)。

下一章预告:第14章 类型系统进阶。record、sealed 类、模式匹配、Optional——现代 Java 的类型安全武器库,也是全书的收官章。